

Zero forcing in the generalized Petersen graphs $P(n, 4)$

Wenlin Zhang and Haobo Ma

8 October 2026 (revised note)

Abstract

We give a computer-assisted proof that $Z(P(n, 4)) = 10$ for every $n \geq 18$, and determine the nine smaller admissible cases. The lower bound uses the external boundary of sixteen forcing sources. Deleting a column from a run of nine empty columns reduces the required vertex-boundary inequality to the finite range $20 \leq n \leq 108$. The cases $n = 18, 19$ are settled by forts. The finite lower bounds and the small forcing witnesses are verified by the accompanying reproducibility package, with a manifest of SHA-256 hashes and separately checked propositional proofs.

The graph $P(n, 4)$ has vertices u_i, v_i , indexed by $\mathbb{Z}/n\mathbb{Z}$, and edges $u_i u_{i+1}$, $u_i v_i$, and $v_i v_{i+4}$. We assume $n \geq 9$, so that these graphs are simple and cubic. In zero forcing, a black vertex with exactly one white neighbor colors that neighbor black. A set is zero forcing if this rule colors every vertex black; $Z(G)$ is the least size of such a set.

Theorem 1. *For every $n \geq 18$, $Z(P(n, 4)) = 10$. For the remaining admissible values,*

n	9	10	11	12	13	14	15	16	17
$Z(P(n, 4))$	6	6	7	6	8	8	9	8	9.

In particular, eighteen is the least eventual stabilization threshold.

Rashidi, Shajareh Poursalavati, and Tavakkoli [1, Theorem 2.2] proved the general upper bound $Z(P(n, k)) \leq 2k + 2$. Krishnan's correction [2] concerns $k = 3$; its conclusion leaves the stabilization threshold and uniform lower bound for $k \geq 4$ open. It does not state a separate conjecture for $k = 4$. We use the first-forcers method from our earlier note *A uniform lower bound for zero forcing in $P(n, 3)$* [3], by Wenlin Zhang and Haobo Ma, dated 6 October 2026 and sent to Krishnan. That note proves his Conjecture 5, namely $Z(P(n, 3)) = 8$ for every $n \geq 13$. Here we use sixteen sources in place of ten. The upper bound below is an explicit instance of the known general construction.

1 An explicit ten-vertex forcing set

For $n \geq 11$, start with $S = \{u_0, \dots, u_9\}$ black. The following sequence is legal:

$$u_i \longrightarrow v_i \quad (1 \leq i \leq 8), \quad v_4 \longrightarrow v_0, \quad v_5 \longrightarrow v_9.$$

The first eight sources have both outer neighbors black. The source v_4 has black neighbors u_4, v_8 and white neighbor v_0 ; similarly v_5 has black neighbors u_5, v_1 and white neighbor v_9 . Thus both vertices in each of columns zero through nine are black.

For $j = 10, \dots, n - 1$, perform

$$u_{j-1} \longrightarrow u_j, \quad v_{j-4} \longrightarrow v_j.$$

The induction invariant before stage j is that both vertices in columns $0, \dots, j - 1$ are black, and every vertex in columns $j, \dots, n - 1$ is still white. Thus the two sources are black and both targets are white. In the first force, the other neighbors are u_{j-2}, v_{j-1} ; in the second, they are u_{j-4}, v_{j-8} . These four vertices lie in earlier columns and are black. The first force leaves v_j white, so the second is legal too. The two forces establish the invariant for the next stage. This colors every vertex. For $n = 9, 10$, taking all outer vertices gives a zero forcing set of size n . Consequently $Z(P(n, 4)) \leq 10$ for every admissible n .

2 The lower bound from sixteen forcing sources

For a vertex set X , write

$$\partial X = \{w \notin X : w \text{ has a neighbor in } X\}$$

for its external vertex boundary.

Lemma 2. *In a finite graph, if a zero forcing set S performs at least p forces, the sources X of its first p forces satisfy $|X| = p$ and $|\partial X| \leq |S|$.*

Proof. A vertex cannot force twice: after it forces, all its neighbors are black. After p forces the black set B has size $|S| + p$, contains X , and contains every neighbor of X . Therefore $\partial X \subseteq B \setminus X$, whose size is $|S|$. $\square$

Lemma 3. *If $n \geq 20$ and $X \subseteq V(P(n, 4))$ has sixteen vertices, then $|\partial X| \geq 10$.*

Proof. Suppose $|\partial X| \leq 9$. Let C be the set of occupied columns, meaning those containing at least one vertex of X , and put $c = |C|$. In each occupied column, every vertex outside X belongs to ∂X by its spoke. Hence

$$2c - 16 \leq |\partial X| \leq 9, \quad c \leq 12.$$

If $n > 108$, some run of empty columns has length at least nine. Indeed, if each of the c cyclic gaps contained at most eight empty columns, then $n \leq c + 8c \leq 108$.

Rotate the empty run to columns $0, \dots, 8$, and delete column four. Define a bijection on the surviving column indices by

$$f : \{0, \dots, n-1\} \setminus \{4\} \longrightarrow \{0, \dots, n-2\}, \quad f(i) = \begin{cases} i, & 0 \leq i \leq 3, \\ i-1, & 5 \leq i \leq n-1. \end{cases}$$

Extend f to vertices by preserving the layer, and put $X' = f(X)$. Every occupied index satisfies $9 \leq i \leq n-1$. For each $d \in \{-4, -1, 1, 4\}$, put $q = i + d$. Then $5 \leq q \leq n+3$, so the deleted column is never a neighbor index. If $q < n$, both i and q are above four and

$$f(q) = q - 1 = f(i) + d.$$

If $q \geq n$, its old residue $q - n$ is in $\{0, 1, 2, 3\}$, and

$$f(q \bmod n) = q - n = (f(i) + d) \bmod (n-1).$$

These are all cases: $d = -1, -4$ never wraps, $d = 1$ wraps only at $i = n-1$, and $d = 4$ wraps at $i = n-4, \dots, n-1$, to old columns $0, \dots, 3$. In particular the old edge from column $n-1$ to column three is preserved.

Conversely, an occupied image index $j = f(i)$ lies in $\{8, \dots, n-2\}$. For the same four offsets, $4 \leq j+d \leq n+2$. A nonwrapping neighbor $j+d \geq 4$ lifts to old column $j+d+1 = i+d$; a wrapping neighbor has residue in $\{0, 1, 2, 3\}$ and lifts to that same residue. Thus no new neighbor is introduced at the deletion. This is where the full nine-column empty buffer is used: old neighbors avoid column four, and new neighbors have exactly the indicated lifts. Spokes are preserved as well. For each $x \in X$, the map therefore carries its three neighbors bijectively onto the three neighbors of $f(x)$.

Writing $N[X] = X \cup \bigcup_{x \in X} N(x)$, we obtain

$$f(N[X]) = N[X'], \quad f(X) = X', \quad f(\partial X) = f(N[X] \setminus X) = N[X'] \setminus X' = \partial X'.$$

The injectivity of f gives $|X'| = 16$ and $|\partial X'| = |\partial X|$. Repeat until the circumference is at most 108. Starting above 108 this procedure stops at 108, so it cannot introduce a smaller exceptional circumference. It remains to exclude the original conditions for $20 \leq n \leq 108$. All 89 cases are excluded by the accompanying package (MANIFEST.sha256); Section 3 specifies the formulas and checks. This completes the proof. $\square$

If $n \geq 20$ and S had at most nine vertices, its forcing process would have at least $2n - 9 \geq 31$ forces. Applying Lemma 2 with $p = 16$ would contradict Lemma 3. Thus $Z(P(n, 4)) \geq 10$ for $n \geq 20$.

The boundary inequality does require a threshold: in $P(19, 4)$ the set

$$X = \{u_0, \dots, u_9, v_3, v_4, v_5, v_6, v_{10}, v_{18}\}$$

has sixteen vertices and boundary $\{u_{10}, u_{18}, v_0, v_1, v_2, v_7, v_8, v_9, v_{14}\}$, of size nine. This explains the separate fort arguments for $n = 18, 19$.

3 Finite verification

The accompanying directory `package/` contains the complete finite evidence for this note. Its `MANIFEST.sha256` identifies the sources, graph data, all 100 CNFs, and compressed binary DRAT proofs by SHA-256 hashes. The computational steps below are verified by this package. They form part of the proof of Lemma 3.

For each $20 \leq n \leq 108$, a Boolean variable x_w records whether $w \in X$. For $20 \leq n \leq 31$, a variable z_w records whether w is in the closed neighborhood of X . For $32 \leq n \leq 108$, z_w instead records membership in ∂X . The respective equivalences are

$$z_w \longleftrightarrow x_w \vee \bigvee_{t \sim w} x_t, \quad \text{or} \quad z_w \longleftrightarrow \neg x_w \wedge \bigvee_{t \sim w} x_t.$$

The cardinality conditions are $\sum_w x_w = 16$ and, respectively, $\sum_w z_w \leq 25$ or $\sum_w z_w \leq 9$. They are equivalent to the boundary conditions because the closed neighborhood is the disjoint union of X and ∂X . We also impose $x_{u_0} \vee x_{v_0}$, which is valid after rotating any nonempty X . The equivalences are expanded directly into clauses, and the cardinalities use sequential counters [4]. The accompanying package verifies that every formula is unsatisfiable: it regenerates and byte-compares each CNF, audits its semantics, and checks the corresponding binary DRAT proof with DRAT-trim [5].

For the smaller zero forcing numbers, a nonempty set F is a *fort* if every vertex outside F has zero or at least two neighbors in F . Every zero forcing set meets F : the first force into a disjoint fort would have exactly one neighbor in it, contradicting this condition. For each $9 \leq n \leq 19$, finite fort families exclude seed sets of size at most one less than the asserted value.

Here the Boolean variables s_w record the seeds, and every fort adds the clause $\bigvee_{w \in F} s_w$. The simultaneous column rotations $i \mapsto i + a$ and reflections $i \mapsto a - i$ on both layers are automorphisms: they preserve spokes and send the same-layer differences ± 1 and ± 4 to themselves. Under this group the directed edges have four orbits: outer edges, inner edges, spokes directed from outer to inner, and spokes directed from inner to outer. Rotating the source to column zero and reflecting when necessary gives the four representatives

$$u_0 \rightarrow v_0, \quad u_0 \rightarrow u_1, \quad v_0 \rightarrow u_0, \quad v_0 \rightarrow v_4.$$

For each case, the source and its other two neighbors must be seeds, and the target must not be a seed. The formula requires at least one case and at most the specified number of seeds. A zero forcing set of size at most the specified bound is not the full vertex set; hence it must admit an initial force, since otherwise the process cannot start. Normalizing that directed edge gives one of the four cases. Thus every candidate zero forcing set satisfies a normalized formula. The package checks every listed set directly against the fort condition and checks all eleven UNSAT proofs. This proves the finite lower bounds, including ten at $n = 18, 19$.

For $9 \leq n \leq 17$, the following seed sets attain the stated values. Complete, ordered forcing sequences are printed in Appendix A and verified by the package.

n	zero forcing set
9	$u_0, u_1, u_2, u_3, u_8, v_4$
10	$u_0, u_1, u_2, u_3, u_9, v_4$
11	$u_0, \dots, u_5, u_{10}$
12	$u_0, u_1, u_2, u_{11}, v_4, v_5$
13	$u_0, u_1, u_2, u_3, u_5, u_6, u_7, u_{12}$
14	$u_0, \dots, u_5, u_8, u_{13}$
15	$u_0, \dots, u_7, u_{14}$
16	$u_0, \dots, u_6, u_{15}$
17	$u_0, \dots, u_7, u_{16}$.

Together with these witnesses and the uniform ten-vertex construction, the verified lower bounds prove Theorem 1.

Reproducing the finite proof. The file `generate.py` deterministically emits the boundary formulas and the fort formulas from the stored vertex lists and clause-order data. The separate `semantic_check.py` rebuilds $P(n, 4)$ from the direct neighbor formulas. It validates every neighborhood clause, every sequential counter grid, the rotation clause, the four first-force cases, and every fort-hitting clause; it also checks that each fort is nonempty and satisfies the fort condition. It checks each forcing step against the current black set, and checks the buffered-deletion identities. This checker imports neither the generator nor the SAT encoding library.

The original proofs were generated by CaDiCaL 1.9.5 through `python-sat` 1.9.dev15. The package includes that pinned distribution's source archive and the producing-run generator sources. DRAT-trim is pinned to source revision `2e3b2dc` (the complete revision and source hash are recorded in `VERSIONS.json`); its source and license are included. Verification needs Python 3.11 or later with its standard library and a C compiler, and runs offline. From the accompanying `package/` directory, execute

```
./verify_all --log-dir ../verification-logs
```

This command checks the manifest, regenerates all CNFs, compares their bytes with the archived files, performs the semantic and forcing checks, builds DRAT-trim using `cc -O2`, decompresses each proof, checks its raw hash, and runs `drat-trim CNF PROOF -i -t 600`. Each of the 100 checks must print `s VERIFIED`; the driver then prints `PASS ALL`. The recorded complete run is supplied alongside the package. Exact setup and fresh proof-generation commands are in `README.md`. The smaller `package-lite/` omits DRAT files and includes their regeneration commands; its semantic-only check does not establish UNSAT.

A Explicit small-case forcing sequences

For each row below, start from the seed set in Section 3 and perform the arrows in the printed order, reading left to right and then top to bottom. Every source is black and has exactly the indicated single white neighbor. The package stores the same sequences with vertex codes $u_i = i$, $v_i = n + i$.

$n = 9$.

$$\begin{array}{cccccc} u_0 \rightarrow v_0 & u_1 \rightarrow v_1 & u_2 \rightarrow v_2 & v_0 \rightarrow v_5 & v_1 \rightarrow v_6 \\ v_2 \rightarrow v_7 & v_5 \rightarrow u_5 & v_6 \rightarrow u_6 & u_5 \rightarrow u_4 & u_6 \rightarrow u_7 \\ & & u_8 \rightarrow v_8 & v_7 \rightarrow v_3 & \end{array}$$

$n = 10$.

$$\begin{array}{cccccc} u_0 \rightarrow v_0 & u_1 \rightarrow v_1 & u_2 \rightarrow v_2 & v_0 \rightarrow v_6 & v_2 \rightarrow v_8 \\ v_4 \rightarrow u_4 & v_6 \rightarrow u_6 & v_8 \rightarrow u_8 & u_3 \rightarrow v_3 & u_4 \rightarrow u_5 \\ & & u_5 \rightarrow v_5 & u_6 \rightarrow u_7 & u_7 \rightarrow v_7 & u_9 \rightarrow v_9 \end{array}$$

$n = 11$.

$$\begin{array}{cccccc} u_0 \rightarrow v_0 & u_1 \rightarrow v_1 & u_2 \rightarrow v_2 & u_3 \rightarrow v_3 & u_4 \rightarrow v_4 \\ v_0 \rightarrow v_7 & v_3 \rightarrow v_{10} & v_4 \rightarrow v_8 & v_7 \rightarrow u_7 & v_8 \rightarrow u_8 \\ v_{10} \rightarrow v_6 & u_7 \rightarrow u_6 & u_8 \rightarrow u_9 & u_9 \rightarrow v_9 & v_1 \rightarrow v_5 \end{array}$$

$n = 12$.

$$\begin{array}{cccccc} u_0 \rightarrow v_0 & u_1 \rightarrow v_1 & v_0 \rightarrow v_8 & v_1 \rightarrow v_9 & v_4 \rightarrow u_4 \\ v_5 \rightarrow u_5 & v_8 \rightarrow u_8 & v_9 \rightarrow u_9 & u_4 \rightarrow u_3 & u_5 \rightarrow u_6 \\ u_8 \rightarrow u_7 & u_9 \rightarrow u_{10} & u_{10} \rightarrow v_{10} & u_{11} \rightarrow v_{11} & u_2 \rightarrow v_2 \\ & & u_3 \rightarrow v_3 & u_6 \rightarrow v_6 & u_7 \rightarrow v_7 \end{array}$$

$n = 13$.

$$\begin{array}{cccccc} u_0 \rightarrow v_0 & u_1 \rightarrow v_1 & u_2 \rightarrow v_2 & u_6 \rightarrow v_6 & v_2 \rightarrow v_{11} \\ v_6 \rightarrow v_{10} & v_{10} \rightarrow u_{10} & v_1 \rightarrow v_5 & v_5 \rightarrow v_9 & v_9 \rightarrow u_9 \\ u_5 \rightarrow u_4 & u_9 \rightarrow u_8 & u_{10} \rightarrow u_{11} & u_{12} \rightarrow v_{12} & v_0 \rightarrow v_4 \\ & & v_4 \rightarrow v_8 & v_{11} \rightarrow v_7 & v_{12} \rightarrow v_3 \end{array}$$

$n = 14$.

$$\begin{array}{cccccc} u_0 \rightarrow v_0 & u_1 \rightarrow v_1 & u_2 \rightarrow v_2 & u_3 \rightarrow v_3 & u_4 \rightarrow v_4 \\ v_0 \rightarrow v_{10} & v_4 \rightarrow v_8 & v_8 \rightarrow v_{12} & v_{12} \rightarrow u_{12} & u_{12} \rightarrow u_{11} \\ u_{13} \rightarrow v_{13} & v_2 \rightarrow v_6 & v_3 \rightarrow v_7 & v_6 \rightarrow u_6 & v_{10} \rightarrow u_{10} \\ v_{13} \rightarrow v_9 & u_5 \rightarrow v_5 & u_6 \rightarrow u_7 & u_8 \rightarrow u_9 & u_{11} \rightarrow v_{11} \end{array}$$

$n = 15$.

$$\begin{array}{cccccc}
u_0 \rightarrow v_0 & u_1 \rightarrow v_1 & u_2 \rightarrow v_2 & u_3 \rightarrow v_3 & u_4 \rightarrow v_4 & \\
u_5 \rightarrow v_5 & u_6 \rightarrow v_6 & v_0 \rightarrow v_{11} & v_1 \rightarrow v_{12} & v_2 \rightarrow v_{13} & \\
v_4 \rightarrow v_8 & v_5 \rightarrow v_9 & v_6 \rightarrow v_{10} & v_8 \rightarrow u_8 & v_9 \rightarrow u_9 & \\
v_{12} \rightarrow u_{12} & v_{13} \rightarrow u_{13} & u_7 \rightarrow v_7 & u_9 \rightarrow u_{10} & u_{10} \rightarrow u_{11} & \\
& & u_{14} \rightarrow v_{14} & & &
\end{array}$$

$n = 16$.

$$\begin{array}{cccccc}
u_0 \rightarrow v_0 & u_1 \rightarrow v_1 & u_2 \rightarrow v_2 & u_3 \rightarrow v_3 & u_4 \rightarrow v_4 & \\
u_5 \rightarrow v_5 & v_0 \rightarrow v_{12} & v_1 \rightarrow v_{13} & v_4 \rightarrow v_8 & v_5 \rightarrow v_9 & \\
v_8 \rightarrow u_8 & v_9 \rightarrow u_9 & v_{12} \rightarrow u_{12} & v_{13} \rightarrow u_{13} & u_8 \rightarrow u_7 & \\
u_9 \rightarrow u_{10} & u_{12} \rightarrow u_{11} & u_{13} \rightarrow u_{14} & u_{14} \rightarrow v_{14} & u_{15} \rightarrow v_{15} & \\
& & v_2 \rightarrow v_6 & v_3 \rightarrow v_7 & v_6 \rightarrow v_{10} & v_7 \rightarrow v_{11}
\end{array}$$

$n = 17$.

$$\begin{array}{cccccc}
u_0 \rightarrow v_0 & u_1 \rightarrow v_1 & u_2 \rightarrow v_2 & u_3 \rightarrow v_3 & u_4 \rightarrow v_4 & \\
u_5 \rightarrow v_5 & u_6 \rightarrow v_6 & v_0 \rightarrow v_{13} & v_1 \rightarrow v_{14} & v_2 \rightarrow v_{15} & \\
v_4 \rightarrow v_8 & v_5 \rightarrow v_9 & v_6 \rightarrow v_{10} & v_9 \rightarrow u_9 & v_{10} \rightarrow u_{10} & \\
v_{13} \rightarrow u_{13} & v_{14} \rightarrow u_{14} & u_9 \rightarrow u_8 & u_{10} \rightarrow u_{11} & u_{13} \rightarrow u_{12} & \\
u_{14} \rightarrow u_{15} & u_{16} \rightarrow v_{16} & v_3 \rightarrow v_7 & v_7 \rightarrow v_{11} & v_8 \rightarrow v_{12} &
\end{array}$$

References

- [1] S. Rashidi, N. Shajareh Poursalavati, and M. Tavakkoli, *Computing the zero forcing number for generalized Petersen graphs*, J. Algebra Comb. Discrete Struct. Appl. 7 (2020), no. 2, 183–193. doi:10.13069/jacodesmath.729465.
- [2] A. Krishnan, *A correction to the Zero Forcing Number of the Generalized Petersen Graphs $P(n, 3)$* , arXiv:2607.19412v1 (2026), Introduction and Section 5.
- [3] W. Zhang and H. Ma, *A uniform lower bound for zero forcing in $P(n, 3)$* , mathematical note, 6 October 2026; communicated to A. Krishnan. Proves Conjecture 5 of [2]: $Z(P(n, 3)) = 8$ for $n \geq 13$.
- [4] C. Sinz, *Towards an optimal CNF encoding of Boolean cardinality constraints*, Principles and Practice of Constraint Programming, LNCS 3709 (2005), 827–831. doi:10.1007/11564751_73.
- [5] M. Heule and contributors, *DRAT-trim*, Source revision and SHA-256 hash in the accompanying VERSIONS.json. The source and its license are included with the supporting files.